

General Templates

Table of Contents

Coordinate Compression.....	1
Coordinate Compression - AmR's Template.....	2
Double Compare.....	3
Line To Array (stringstream).....	3
Convert 2D to 1D.....	3
Anti-hash-collision wrapper (unordered).....	4
Interactive Problems.....	4
Sorting.....	5
Custom Comparator Operator.....	5
Custom sort.....	5
Sort in struct.....	6
Lamda Function.....	6
Tuples in C++.....	7
Back Tracking.....	8
Iterate through all subsets using bitmasks.....	13

Coordinate Compression

```
/*
 * Template: Coordinate Compression (General Workflow Guide)
 *
 * WHEN TO USE COORDINATE COMPRESSION:
 * - When array values or query points are large (> 1e6) or negative, but
only
 * their relative order/ranks matter.
 * - Maps arbitrary large values into a contiguous range [0, Unique_Count -
1].
 *
 * GENERAL USAGE WORKFLOW:
 *
 * 1- Gather All Values to Compress:
 *   - Static Array: Pass the original array `a` to `comp.init(a)`.
 *   - Dynamic/Queries (Point Updates or Value Range Queries):
 *     Collect both array elements AND any new values from queries into a
single
 *     `vector<int> all_vals`, then pass it to `comp.init(all_vals)`.
 *
 * 2- Allocate Data Structure Size:
 *   - Size frequency arrays / Data Structures using the unique count:
 *     `int max_val = comp.size();`
 *     `vis.assign(max_val + 5, 0);` OR `FenwickTree tree(max_val + 5);`
 *
 * 3- Transform Raw Values ↔ Compressed Values:
 *   - Compress (Value → 0-based Rank Index):
 *     `int c_id = comp.compress(val);` // O(Log N)
 *   - Decompress (0-based Rank Index → Original Value):
 *     `int orig_val = comp.decompress(c_id);` // O(1)
 *
 * 4- Data Structure Operations:
 *   - Use `c_idx` whenever accessing array indices, Fenwick Tree, or
SegTree nodes.
 *   - Use `orig_val` whenever evaluating mathematical conditions or
printing results.
 */
```

```
struct CoordinateCompression
{
    vector<int> vals;

    CoordinateCompression() {}
    CoordinateCompression(const vector<int> &a) { init(a); }
    void init(const vector<int> &a)
```

```

{
    vals = a;
    sort(vals.begin(), vals.end());
    vals.erase(unique(vals.begin(), vals.end()), vals.end());
}
int compress(int val) const
{
    auto it = lower_bound(vals.begin(), vals.end(), val);
    return it - vals.begin();
}
int decompress(int comp_idx) const
{
    return vals[comp_idx];
}
int size() const
{
    return (int)(vals.size());
}
};

```

Coordinate Compression - AmR's Template

```

struct coordinateCompression {
private:
    vector<ll> init;

    void compress(vector<ll>& v) {
        sort(v.begin(), v.end());
        v.erase(unique(v.begin(), v.end()), v.end());
    }

public:
    coordinateCompression(const vector<ll>& v) {
        init = v;
        compress(init);
    }

    int index(ll val) const {
        return lower_bound(init.begin(), init.end(), val) - init.begin();
    }

    ll initVal(int idx) const {
        return init[idx];
    }
}

```

```
int size() const {  
    return init.size();  
}  
};
```

Double Compare

```
int double_comp(double a, double b)  
{  
    if(fabs1(a - b) < 1e-9){  
        return 0;  
    }else{  
        if(a < b) return -1;  
        else return 1;  
    }  
}
```

Line To Array (stringstream)

```
vector<int> parseLineToArray(const string &line)  
{  
    stringstream ss(line);  
    int num;  
    vector<int> res;  
    while (ss >> num)  
    {  
        res.push_back(num);  
    }  
    return res;  
}
```

Convert 2D to 1D

Note: 1-based

```
index = row * num_cols + col
```

Anti-hash-collision wrapper (unordered)

Note: only for integers and long long

```
struct custom_hash
{
    static uint64_t splitmix64(uint64_t x)
    {
        x += 0x9e3779b97f4a7c15;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
        x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
        return x ^ (x >> 31);
    }
    size_t operator()(uint64_t x) const
    {
        static const uint64_t FIXED_RANDOM =
chrono::steady_clock::now().time_since_epoch().count();
        return splitmix64(x + FIXED_RANDOM);
    }
};
unordered_map<ll, ll, custom_hash> s;
unordered_set<int, custom_hash> ss;
```

Interactive Problems

The input will contain responses to your queries — strings "<" and ">=". When your program will guess the number print "! x", where x is the answer and terminate your program.

```
string ask(int x) {
    cout << x << nl;
    cout.flush();
    string sign;
```

```

    cin >> sign;
    return sign;
}
void solve() {
    int l = 1, r = 1000000, ans = 0;
    while(l <= r) {
        int mid = (l + r) / 2;
        if(ask(mid) == ">=") {
            l = mid + 1;
            ans = mid;
        }

        else r = mid - 1;
    }
    cout << "! " << ans;
}

```

Examples

input

```

>=
<
>=
>=
<
<

```

output

```

8
20
11
12
13
16
! 12

```

Sorting

Custom Comparator Operator

```

struct Element {
    int val;
    int weight;
    int id;

    // Custom Comparator Operator
    bool operator<(const Element &other) const {
        if (weight != other.weight) {
            return weight > other.weight; // تنازلي حسب weight
        }
        if (val != other.val) {
            return val < other.val; // تصاعدي حسب val
        }
        return id < other.id; // لكسر التعادل id تصاعدي حسب
    }
};

```

Custom sort

```
sort(v.begin(), v.end(), [](const pair<int, int> &a, const pair<int, int>
&b) {
    if (a.first != b.first)
        return a.first > b.first; // تنازلي
    return a.second < b.second;    // تصاعدي
});
```

```
struct CustomCmp {
    bool operator()(const pair<int, int> &a, const pair<int, int> &b) const
    {
        if (a.first != b.first) return a.first < b.first;
        return a.second > b.second;
        // in priority queue : we should reverse sorting
        // a.second > b.second in priority queue is Min-Heap
    }
};

set<pair<int, int>, CustomCmp> st;
```

```
sort(all(x), greater<>()); // == sort(rbegin(), rend());
```

Sort in struct

```
struct ReadyCell {
    ll time;
    int r, c;
    // ترتيب تصاعدي من الصغير للكبير
    bool operator>(const ReadyCell &o) const {
        return time > o.time;
    }
};
```

Ex: `priority_queue<ReadyCell, vector<ReadyCell>, greater<ReadyCell>> ready;`
 sorts in descending order

Lamda Function

```
auto <function_name> = [<type> <parameters>] -> <return_type>
{
```

```

    // return
};

// example one
auto add = [](int a, int b) -> int
{
    return a + b;
};

// example two
auto dfs = [&](auto& self, int u, int p, int d) -> void
{
    depth[u] = d;
    for (int v : adj[u]) {
        if (v != p) {
            self(self, v, u, d + 1); // recursive call
        }
    }
};

```

Tuples in C++

```

vector<tuple<int, string, double>> v;

    v.emplace_back(1, "Ahmed", 90.5);
    v.push_back({2, "Sara", 98.0});

    // القراءة من عنصر محدد (مثلاً أول عنصر 1)
    cout << get<0>(v[0]) << " " << get<1>(v[0]) << "\n"; // 1 Ahmed

    // 2. التعديل على عنصر
    get<2>(v[0]) = 95.0; // لأول طالب score تعديل الى

// -----

tuple<int, string, double> t(1, "Ahmed", 95.5);

    // 1. قراءة القيم
    cout << get<0>(t) << "\n"; // 1
    cout << get<1>(t) << "\n"; // Ahmed
    cout << get<2>(t) << "\n"; // 95.5

    // 2. التعديل على القيم (get بترجع Reference)
    get<1>(t) = "Mohamed";

```

```
get<2>(t) += 2.0;
```

```
cout << get<1>(t) << " -> " << get<2>(t) << "\n"; // Mohamed -> 97.5
```

Back Tracking

```
=====
// 1. N-QUEENS (BITMASK OPTIMIZED)
//
=====
// Problem: Count valid placements of N-Queens on an N x N board.
// Time Complexity: O(N!) with extreme pruning
// Space Complexity: O(N) recursion stack
int n_queens_count(int row, int cols, int diag1, int diag2, int n) {
    if (row == n) return 1;
    int count = 0;
    // Available positions: all bits up to n that are not attacked
    int available = ((1 << n) - 1) & ~(cols | diag1 | diag2);

    while (available) {
        int bit = available & -available; // Extract lowest set bit
        available -= bit;
        count += n_queens_count(row + 1,
                                cols | bit,
                                (diag1 | bit) << 1,
                                (diag2 | bit) >> 1,
                                n);
    }
    return count;
}

//
=====
// 2. COMBINATION SUM II (UNIQUE COMBINATIONS WITH DUPLICATES)
//
=====
// Problem: Generate all unique combinations that sum up to `target` using
// each element at most once.
// Time Complexity: O(2^N)
// Space Complexity: O(N) recursion depth + current path
void combination_sum_unique(int start, int target, vector<int>& a,
vector<int>& current, vector<vector<int>>& results) {
    if (target == 0) {
```

```

        results.push_back(current);
        return;
    }
    for (int i = start; i < (int)a.size(); i++) {
        if (a[i] > target) break; // Pruning (requires array to be sorted)
        if (i > start && a[i] == a[i - 1]) continue; // Skip duplicates at
the same recursion depth

        current.push_back(a[i]);
        combination_sum_unique(i + 1, target - a[i], a, current, results);
        current.pop_back(); // Backtrack
    }
}

//
=====
// 3. UNIQUE PERMUTATIONS WITH DUPLICATES
//
=====
// Problem: Generate all distinct permutations of a multiset of numbers.
// Time Complexity: O(N * N!)
// Space Complexity: O(N)
void unique_permutations(vector<int>& a, vector<bool>& used, vector<int>&
current, vector<vector<int>>& results) {
    if (current.size() == a.size()) {
        results.push_back(current);
        return;
    }
    for (int i = 0; i < (int)a.size(); i++) {
        if (used[i]) continue;
        // If duplicate element has not been used in this branch yet, skip to
avoid duplicate permutations
        if (i > 0 && a[i] == a[i - 1] && !used[i - 1]) continue;

        used[i] = true;
        current.push_back(a[i]);
        unique_permutations(a, used, current, results);
        current.pop_back(); // Backtrack
        used[i] = false;
    }
}

//
=====
// 4. GRID WORD SEARCH (IN-PLACE VISITATION)

```

```
//
=====
// Problem: Determine if a word exists in an N x M grid of Letters
(4-directional, no cell reuse).
// Time Complexity:  $O(N * M * 3^L)$  where  $L = \text{word.length}()$ 
// Space Complexity:  $O(L)$ 
const int dx[] = {0, 0, 1, -1};
const int dy[] = {1, -1, 0, 0};

bool word_search(int r, int c, int idx, const string& word,
vector<vector<char>>& board) {
    if (idx == (int)word.size()) return true;
    int n = board.size(), m = board[0].size();
    if (r < 0 || r >= n || c < 0 || c >= m || board[r][c] != word[idx])
return false;

    char original = board[r][c];
    board[r][c] = '#'; // Mark as visited in-place

    for (int d = 0; d < 4; d++) {
        if (word_search(r + dx[d], c + dy[d], idx + 1, word, board)) {
            board[r][c] = original; // Restore state
            return true;
        }
    }

    board[r][c] = original; // Backtrack
    return false;
}

//
=====
// 5. PARTITION TO K EQUAL SUM SUBSETS
//
=====
// Problem: Partition an array of N elements into K non-empty subsets of
equal sum.
// Time Complexity:  $O(K * 2^N)$  with descending sort pruning
// Space Complexity:  $O(N + K)$ 
bool can_partition_k_subsets(int idx, int k, int target, vector<int>& a,
vector<int>& bucket_sums) {
    if (idx == (int)a.size()) return true;

    for (int i = 0; i < k; i++) {
        if (bucket_sums[i] + a[idx] > target) continue;

```

```

        bucket_sums[i] += a[idx];
        if (can_partition_k_subsets(idx + 1, k, target, a, bucket_sums))
return true;
        bucket_sums[i] -= a[idx]; // Backtrack

        // Crucial pruning: if an empty bucket fails, all other identical
empty buckets will also fail
        if (bucket_sums[i] == 0) break;
    }
    return false;
}

//
=====
// 6. SUDOKU SOLVER (BITMASK OPTIMIZED)
//
=====
// Problem: 9x9 Sudoku Solver with Bitmask validity checks.
// Time Complexity: O(9^empty_cells)
// Space Complexity: O(empty_cells)
int row_mask[9], col_mask[9], box_mask[9];

int get_box_id(int r, int c) {
    return (r / 3) * 3 + (c / 3);
}

bool solve_sudoku(int cell, const vector<pair<int, int>>& empty_cells,
vector<vector<int>>& grid) {
    if (cell == (int)empty_cells.size()) return true;

    auto [r, c] = empty_cells[cell];
    int b = get_box_id(r, c);
    int used = row_mask[r] | col_mask[c] | box_mask[b];

    for (int num = 1; num <= 9; num++) {
        if (!(used & (1 << num))) {
            // Apply choice
            row_mask[r] |= (1 << num);
            col_mask[c] |= (1 << num);
            box_mask[b] |= (1 << num);
            grid[r][c] = num;

            if (solve_sudoku(cell + 1, empty_cells, grid)) return true;

            // Backtrack
            grid[r][c] = 0;
        }
    }
}

```

```

        row_mask[r] &= ~(1 << num);
        col_mask[c] &= ~(1 << num);
        box_mask[b] &= ~(1 << num);
    }
}
return false;
}

//
=====
// 7. PARTITION ARRAY INTO 2 ARITHMETIC-PROGRESSION-FREE SUBSETS
// (BACKTRACKING)
//
=====
// Problem: Partition array into 2 disjoint sets (Set A & Set B) such that
// neither
//         contains any 3-term Arithmetic Progression ( $a[k] - a[j] == a[j] - a[i] \Leftrightarrow 2*a[j] == a[i] + a[k]$ ).
// Technique: Pre-sort the array so for any 3 elements  $i < j < k$ , the AP
// condition is  $2*a[j] == a[i] + val$ .
//         Before adding `val` to a set, verify it doesn't form an AP with
// any previous pair (i, j).
// Time Complexity:  $O(2^N * N)$  in the worst case (significantly faster with
// AP pruning).
// Space Complexity:  $O(N)$ 
bool check_ap_violation(const vector<int>& subset, int val) {
    int sz = subset.size();
    if (sz < 2) return false;
    for (int i = 0; i < sz; i++) {
        for (int j = i + 1; j < sz; j++) {
            // Since elements are sorted:  $subset[i] < subset[j] < val$ 
            if (2LL * subset[j] == (long long)subset[i] + val) {
                return true; // Found an AP of length 3: (subset[i],
subset[j], val)
            }
        }
    }
    return false;
}

bool partition_ap_free(int idx, const vector<int>& a, vector<int>& setA,
vector<int>& setB) {
    if (idx == (int)a.size()) return true;

    // Option 1: Try putting a[idx] into Set A
    if (!check_ap_violation(setA, a[idx])) {

```

```

        setA.push_back(a[idx]);
        if (partition_ap_free(idx + 1, a, setA, setB)) return true;
        setA.pop_back(); // Backtrack
    }

    // Option 2: Try putting a[idx] into Set B
    if (!check_ap_violation(setB, a[idx])) {
        setB.push_back(a[idx]);
        if (partition_ap_free(idx + 1, a, setA, setB)) return true;
        setB.pop_back(); // Backtrack
    }

    return false;
}

//
=====
// 8. BEAUTIFUL ARRAY / AP-FREE PERMUTATION (DIVIDE & CONQUER / RECURSION)
//
=====
// Problem: Rearrange numbers [1...N] (or any array) into a permutation with
// NO 3-element APs
//           (i.e., there is no  $i < j < k$  such that  $2 * A[j] == A[i] + A[k]$ ).
// Technique: Parity Separation: If left half contains ONLY odd numbers and
// right half contains
//           ONLY even numbers, any pair (odd, even) has an odd sum ( $A[i] + A[k]$ ), which can NEVER
//           equal  $2 * A[j]$  (even). Recursively apply this logic ( $2*x - 1$ 
// for odds,  $2*x$  for evens).
// Time Complexity:  $O(N \log N)$ 
// Space Complexity:  $O(N)$ 
vector<int> generate_ap_free_permutation(int n) {
    if (n == 1) return {1};

    // Divide into Odds and Evens
    vector<int> odds = generate_ap_free_permutation((n + 1) / 2);
    vector<int> evens = generate_ap_free_permutation(n / 2);

    vector<int> result;
    result.reserve(n);

    // Map:  $x \rightarrow 2*x - 1$  (Generates all odd numbers in [1...n])
    for (int x : odds) {
        result.push_back(2 * x - 1);
    }
    // Map:  $x \rightarrow 2*x$  (Generates all even numbers in [1...n])

```

```
for (int x : evens) {
    result.push_back(2 * x);
}

return result;
}
```

Iterate through all subsets using bitmasks

```
for (int mask = 0; mask < (1 << n); ++mask) {
    for (int i = 0; i < n; ++i) {
        if ((mask & (1 << i)) != 0) {
            // Element i is included in the current subset
        }
    }
}
```